

AUTOMATA LANGUAGES AND COMPUTATION

SYLLABUS

UNIT – I

Finite automata and Regular expression: Deterministic and Non-Deterministic Finite Automata, Finite automata with ϵ -moves, Regular expression – Equivalence of NFA and DFA ,Two- way finite automata, Moore machine and Mealy machines , Applications of finite automata.

INDEX

SL.NO.	CONTENTS	PAGE NO.
1.	FINITE AUTOMATA	
2.	TYPES OF FINITE AUTOMATA	
2.1.	DETERMINISTIC FINITE AUTOMATA	
2.2.	NON-DETERMINISTIC FINITE AUTOMATA	
2.3.	CONVERSION OF NFA TO DFA	
3.	FINITE AUTOMATA WITH ϵ -MOVES	
4.	CONVERSION OF NFA WITH ϵ – TRANSITION TO NFA WITHOUT ϵ -TRANSITION	
5.	EQUIVALENCE OF NFA AND DFA	
6.	REGULAR EXPRESSION	
7.	REGULAR EXPRESSION THEOREM	
8.	TWO-WAY FINITE AUTOMATA	
9.	FINITE AUTOMATA WITH OUTPUT	
10.	CONVERSION OF MOORE TO MEALY MACHINE	
11.	APPLICATIONS OF FINITE AUTOMATA	

UNIT –I

1. FINITE AUTOMATA AND REGULAR EXPRESSION

1.1 FINITE AUTOMATA

Finite automata is a mathematical model of a system with discrete inputs and outputs .The system can be in any one of finite number of states and the state summarizes the history of inputs and determines the behaviour of the system for subsequent input.

1.1.1 COMPONENTS OF FINITE AUTOMATA

The block diagram of the finite automata contains three components namely,

- 1 .Input tape
2. Reading Head Pointer
3. Finite Control

INPUT TAPE

The input tape is divided into number of squares or cells. Each square contains single symbol or alphabet from the input alphabet Σ . The left end square of the tape contains ϵ and the right end square contains \$ symbol. The absence of the end markers indicate that the tape is of infinite length. The left to right sequence of symbols between the end markers is the string to be processed.

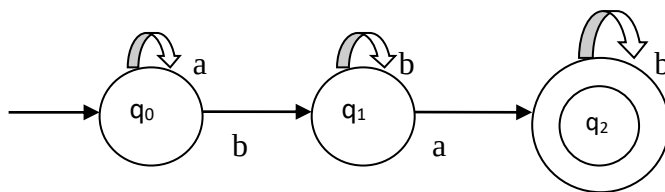
READING HEAD POINTER

The head examines only one square at a time and can move either one square to the left or to the right. We restrict the movement of the reading head pointer only to the right side.

FINITE CONTROL

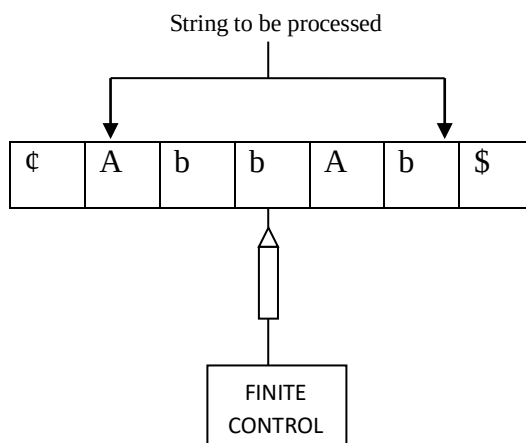
The finite control contains the routines, which instruct the reading head pointer to move from one state to the next state by recognising each and every symbol or alphabet. The transition of reading head pointer from one state to another state by recognising an alphabet is indicated by $\delta(q, a)$.

For E.g.: Consider a Finite Automata with the transition diagram

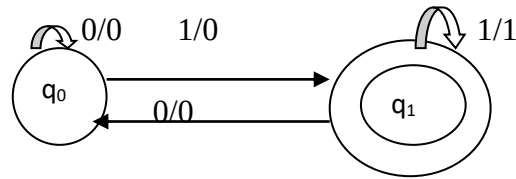


The transition graph or transition diagram is a finite labelled graph in which each vertex or node represent a state and the directed arcs indicates the transition of a state and the edges are labelled with input/output. The initial state is indicated with circle and arrow pointing towards it, and the final state is indicated by concentric circles.

Block diagram of Finite Automata



A transition graph or a transition system is a finite directed graph in which each vertex or node represents a state and the directed edge are labelled with output/input.



Automaton in which the output depends only on the input is called as an automaton without memory. An automaton in which the output depends only on the states of the machine is called as Moore machine. An automaton in which output depends only on the states as well as on the input at any instant of time is called a Mealy machine. Both the Moore and Mealy machine are Finite automaton with output.

2. TYPES OF FINITE AUTOMATA

The finite automata can be divided into

- a) Deterministic Finite Automata (DFA).
- b) Non-deterministic Finite Automata (NFA or NDFA)

2.1 DETERMINISTIC FINITE AUTOMATA

The deterministic finite automata can be represented by 5-tuples. If M is the deterministic finite automata then,

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where,

M is the NFA with ϵ moves.

Q is the finite set of non-empty states.

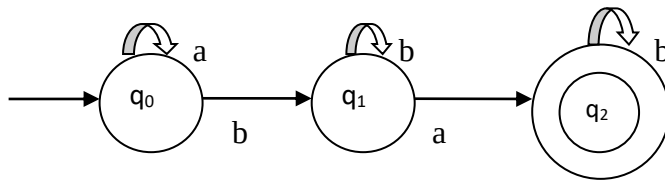
Σ is the finite set of non-empty symbols.

δ is the finite set of transitions.

q_0 is the initial state.

F is the final state.

Consider an example,



Here

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_2\}$$

Transition Table

δ	a	b
$\{q_0\}$	$\{q_0\}$	$\{q_1\}$
$\{q_1\}$	$\{q_2\}$	$\{q_1\}$
$\{q_2\}$	$\emptyset$	$\{q_2\}$

The transition mapping is

$$\delta: Q \times \Sigma \rightarrow Q$$

Where $q_0 \in Q$ and $a \in \Sigma$ and it is mapped on to $q_0 \in Q$ and $q_1 \in Q$.

2.2 NON DETERMINISTIC FINITE AUTOMATA

A non-deterministic finite automaton (NFA) can be represented by 5 tuples namely

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where,

M is the NFA with ϵ moves.

Q is the finite set of non-empty states.

Σ is the finite set of non-empty symbols.

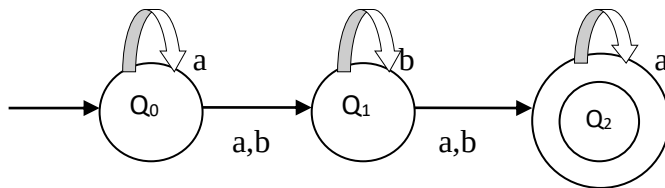
δ is the finite set of transitions.

q_0 is the initial state.

F is the final state.

i.e. " δ " is the transition mapping function from $Q \times \Sigma \rightarrow 2^Q$, where are mapped to subsets of the input individual state.

e.g.:



Where,

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_2\}$$

Transition Table

δ	a	b
q_0	(q_0, q_1)	(q_1)
sq_1	(q_2)	(q_1, q_2)
q_2	(q_2)	Φ

The transition mapping is

$$Q \times \Sigma \rightarrow 2^Q \text{ or } Q'$$

$$\delta(q_0, a) = (q_0, q_1)$$

where $q_0 \in Q$, $a \in \Sigma$ and $(q_0, q_1) \in Q'$ rather than $(q_0, q_1) \in Q$ since,

$$Q' = 2^Q \text{ (subsets of individual states)}$$

$$= \{(q_0), (q_1), (q_2), (q_0, q_1), (q_1, q_2), (q_0, q_2), (q_0, q_1, q_2), \Phi\}$$

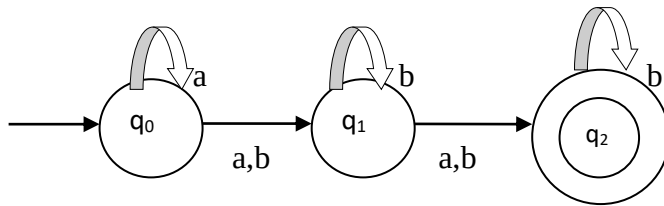
DIFFERENCE BETWEEN NFA AND DFA

The main difference between NFA and DFA is,

- The deterministic finite automata contains only at most one transition from each and every state for each input symbol but the non-deterministic finite automata contains more than one transition for each and every symbol from each and every state.
- The second difference is that DFA has the transition mapping $Q \times \Sigma \rightarrow Q$ where as the NFA contains the transition mapping $Q \times \Sigma \rightarrow 2^Q \text{ or } Q'$.

2.3. CONVERSION OF NFA TO DFA

Consider a NFA with the transition diagram,



$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$F = \{q_2\}$$

Transition Table

δ	a	b
q_0	(q_0, q_1)	(q_1)
q_1	(q_2)	(q_1, q_2)
q_2	Φ	(q_2)

Converting to DFA

The DFA can be expressed during conversion

$$M' = (Q, \Sigma, \delta', q_0, F')$$

Where $Q' = 2^Q$.

$$\Sigma = \{a, b\}$$

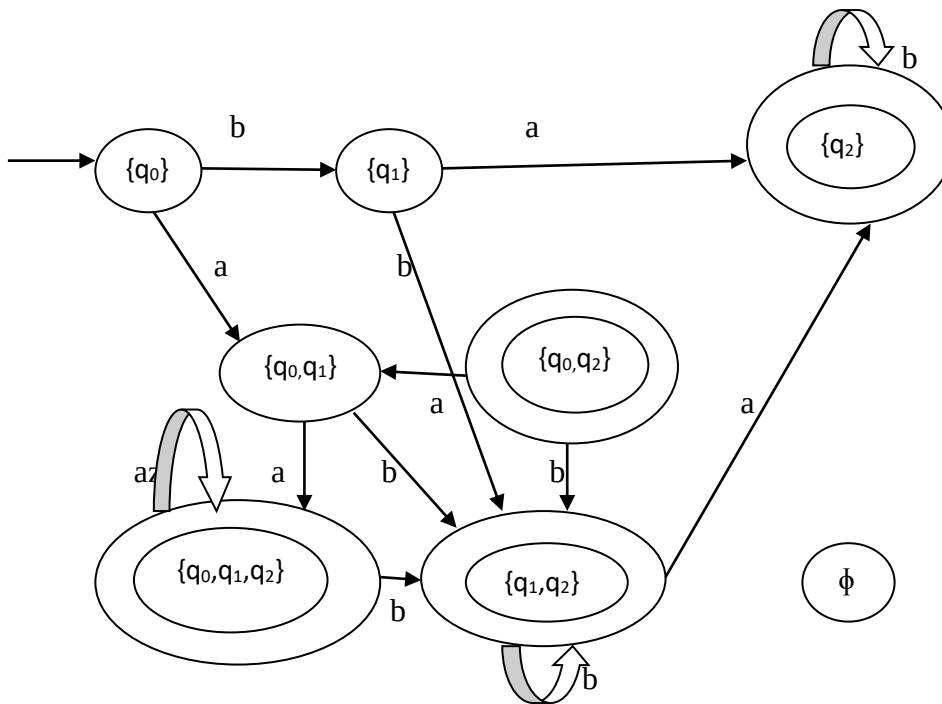
$$q_0 = q_0$$

$$F' = \{\{q_2\}, \{q_1, q_2\}, \{q_0, q_2\}, \{q_0, q_1, q_2\}\}$$

Transition Table

δ'	a	b
$\{q_0\}$	$\{q_0, q_1\}$	$\{q_1\}$
$\{q_1\}$	$\{q_2\}$	$\{q_1, q_2\}$
$\{q_2\}$	$\emptyset$	$\{q_2\}$
$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_1, q_2\}$
$\{q_1, q_2\}$	$\{q_2\}$	$\{q_1, q_2\}$
$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$
$\emptyset$	$\emptyset$	$\emptyset$

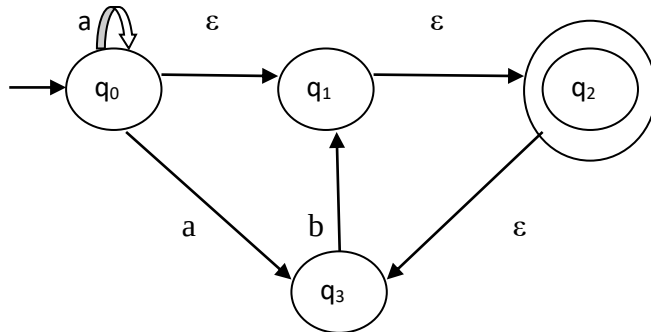
TRANSITION DIAGRAM



3. FINITE AUTOMATA WITH ϵ -MOVES

The ' ϵ ' is a character used to indicate the null string i.e. the string which is used simply for transition from one state to the other without any input. The NFA with ϵ -moves can be shown below:

e.g.



Let us define NFA with ϵ -transition as

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where,

M is the NFA with ϵ moves.

Q is the finite set of non-empty states.

Σ is the finite set of non-empty symbols.

δ is the finite set of transitions.

q_0 is the initial state of NFA.

F is the final state of NFA with ϵ -moves.

The transition mapping is given by

$$Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q \text{ or } Q'.$$

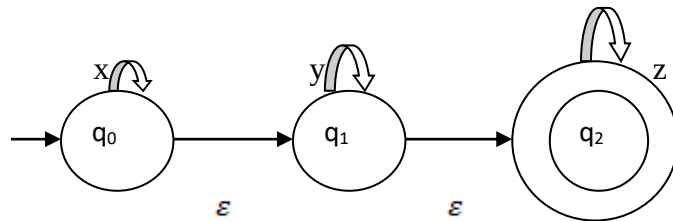
ϵ -closure(q) denotes the set of all states P such that there is a path from q to p labelled ϵ . The

δ is interpreted as follows,

$$\delta(q, \epsilon) = \epsilon\text{-closure}(q).$$

e.g.

Consider finite automata with ϵ -moves



The ϵ -closure for each and every state as follows:

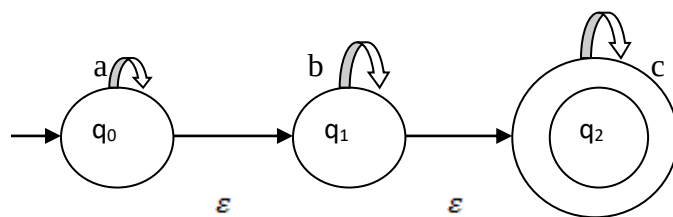
$$\delta'(q_0, \epsilon) = \epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\delta'(q_1, \epsilon) = \epsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\delta'(q_2, \epsilon) = \epsilon\text{-closure}(q_2) = \{q_2\}$$

4. CONVERSION OF NFA WITH ϵ -TRANSITION TO NFA WITHOUT ϵ -TRANSITION

Consider a NFA with ϵ -transition has 5 tuples



$$M = (Q, \Sigma, \delta, q_0, F)$$

Where,

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b, c, \epsilon\}$$

δ is the transition function that maps,

$$Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q \text{ or } Q'.$$

Transition table

Δ	a	b	c	ε
$\{q_0\}$	$\{q_0\}$	$\emptyset$	$\emptyset$	$\{q_1\}$
$\{q_1\}$	$\emptyset$	$\{q_1\}$	$\emptyset$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\emptyset$	$\{q_2\}$	$\emptyset$

$$q_0 = \{q_0\}$$

$$F = \{q_2\}$$

When converting NFA with ε -transition to NFA without ε -transition, one automaton contains

$$M' = (Q, \Sigma, \delta', q_0, F')$$

Where,

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b, c\}$$

δ' is the transition function for NFA without transition, then we first compute the ε -closure of each and every state.

$$\varepsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\varepsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\varepsilon\text{-closure}(q_2) = \{q_2\}$$

The transition function for each and every input symbol can be summarised as:

$$\begin{aligned}
\delta'(q_0, a) &= \epsilon\text{-closure}(\delta(\delta'(q_0, \epsilon), a)) \\
&= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, a)) \\
&= \epsilon\text{-closure}(\delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a)) \\
&= \epsilon\text{-closure}(q_0 \cup \phi \cup \phi) \\
&= \epsilon\text{-closure}(q_0) \\
&= \{q_0, q_1, q_2\}
\end{aligned}$$

$$\begin{aligned}
\delta'(q_0, b) &= \epsilon\text{-closure}(\delta(\delta'(q_0, \epsilon), b)) \\
&= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, b)) \\
&= \epsilon\text{-closure}(\delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b)) \\
&= \epsilon\text{-closure}(\phi \cup q_1 \cup \phi) \\
&= \epsilon\text{-closure}(q_1) \\
&= \{q_1, q_2\}
\end{aligned}$$

$$\begin{aligned}
\delta'(q_0, c) &= \epsilon\text{-closure}(\delta(\delta'(q_0, \epsilon), c)) \\
&= \epsilon\text{-closure}(\delta(\{q_0, q_1, q_2\}, c)) \\
&= \epsilon\text{-closure}(\delta(q_0, c) \cup \delta(q_1, c) \cup \delta(q_2, c)) \\
&= \epsilon\text{-closure}(\phi \cup \phi \cup q_2) \\
&= \epsilon\text{-closure}(q_2) \\
&= \{q_2\}
\end{aligned}$$

$$\begin{aligned}
\delta'(q_1, a) &= \varepsilon\text{-closure}(\delta(\delta'(q_1, \varepsilon), a)) \\
&= \varepsilon\text{-closure}(\delta(\{q_1, q_2\}, a)) \\
&= \varepsilon\text{-closure}(\delta(q_1, a) \cup \delta(q_2, a)) \\
&= \varepsilon\text{-closure}(\emptyset \cup \emptyset) \\
&= \varepsilon\text{-closure}(\emptyset) \\
&= \emptyset
\end{aligned}$$

$$\begin{aligned}
\delta'(q_1, b) &= \varepsilon\text{-closure}(\delta(\delta'(q_1, \varepsilon), b)) \\
&= \varepsilon\text{-closure}(\delta(\{q_1, q_2\}, b)) \\
&= \varepsilon\text{-closure}(\delta(q_1, b) \cup \delta(q_2, b)) \\
&= \varepsilon\text{-closure}(q_1 \cup \emptyset) \\
&= \varepsilon\text{-closure}(q_1) \\
&= \{q_1, q_2\}
\end{aligned}$$

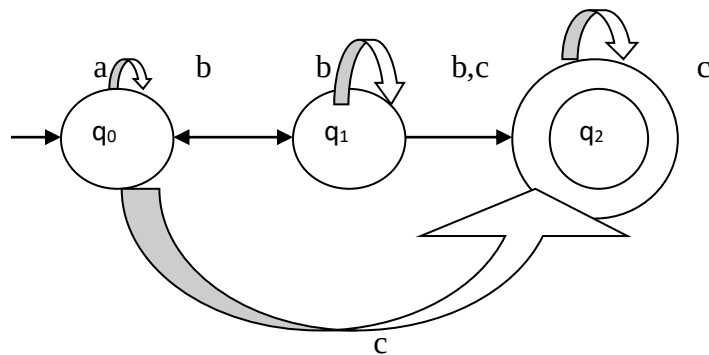
$$\begin{aligned}
\delta'(q_1, c) &= \varepsilon\text{-closure}(\delta(\delta'(q_1, \varepsilon), c)) \\
&= \varepsilon\text{-closure}(\delta(\{q_1, q_2\}, c)) \\
&= \varepsilon\text{-closure}(\delta(q_1, c) \cup \delta(q_2, c)) \\
&= \varepsilon\text{-closure}(\emptyset \cup q_2) \\
&= \varepsilon\text{-closure}(q_2) \\
&= \{q_2\}
\end{aligned}$$

$$\begin{aligned}
\delta'(q_2, a) &= \epsilon\text{-closure}(\delta(\delta'(q_2, \epsilon), a)) \\
&= \epsilon\text{-closure}(\delta(\{q_2\}, a)) \\
&= \epsilon\text{-closure}(\delta(q_2, a)) \\
&= \epsilon\text{-closure}(\emptyset) \\
&= \emptyset
\end{aligned}$$

$$\begin{aligned}
\delta'(q_2, b) &= \epsilon\text{-closure}(\delta(\delta'(q_2, \epsilon), b)) \\
&= \epsilon\text{-closure}(\delta(\{q_2\}, b)) \\
&= \epsilon\text{-closure}(\delta(q_2, b)) \\
&= \epsilon\text{-closure}(\emptyset) \\
&= \emptyset
\end{aligned}$$

$$\begin{aligned}
\delta'(q_2, c) &= \epsilon\text{-closure}(\delta(\delta'(q_2, \epsilon), c)) \\
&= \epsilon\text{-closure}(\delta(\{q_2\}, c)) \\
&= \epsilon\text{-closure}(\delta(q_2, c)) \\
&= \epsilon\text{-closure}(q_2) \\
&= \{q_2\}
\end{aligned}$$

The transition for NFA without ϵ -moves:



5. EQUIVALENCE OF NFA AND DFA

Statement:

Let 'L' be a set accepted by a non-deterministic finite automata then there exists a deterministic finite automaton that accepts "L".

To Prove:

The Language accepted by NFA is equal to the language set accepted by DFA.

i.e. $L(M) = L(M')$

Where M is the NFA and M' is the DFA.

Proof:

Let $M = (Q, \Sigma, \delta, q_0, F)$ be an NFA, accepting L and we can define a DFA

$M' = (Q, \Sigma, \delta', q_0, F')$ as follows .

The states of M' are all the subsets of the set of states of M .

That is, $Q' = 2^Q$

M' will keep track in its states of all the states M could be in at any given time. F' is the set all the states in Q' containing the final state of M . An element of Q' will be denoted by

$[q_1, q_2, q_3, \dots, q_i]$ are in Q .

Observe that, $[q_1, q_2, q_3, \dots, q_i]$ is a single state of DFA corresponding to the states of the NFA. Note $q_0' = [q_0]$

We define

$$\delta'([q_1, q_2, q_3, \dots, q_i], a) = [P_1, P_2, P_3, \dots, P_j]$$

if and only if

$$\delta([q_1, q_2, q_3, \dots, q_i], a) = [P_1, P_2, P_3, \dots, P_j]$$

that is

δ' applied to an element $[q_1, q_2, q_3, \dots, q_i]$ of Q' is computed by applying δ to each state of Q represented by $[q_1, q_2, q_3, \dots, q_i]$ in applying δ to each of $[q_1, q_2, q_3, \dots, q_i]$ and taking “union” we get some new set of states $P_1, P_2, P_3, \dots, P_j$. This new set of states has a representative $P_1, P_2, P_3, \dots, P_j$ in Q' and that element is the value of $\delta'([q_1, q_2, q_3, \dots, q_i], a)$.

It is easy to show by induction for the length of the input string x , that

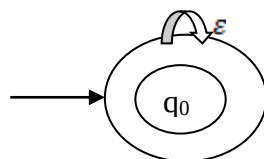
$$\delta'(q_0', x) = [q_1, q_2, q_3, \dots, q_i]$$

if and only if,

$$\delta(q_0, x) = \{q_1, q_2, q_3, \dots, q_i\}$$

Basis

Consider $|x|=0$ then x must be ϵ since $q_0' = [q_0]$



Then

$$\delta'(q_0', \epsilon) = (q_0)$$

if and only if

$$\delta(q_0, \epsilon) = \{q_0\}$$

Hence the result is trivial.

Induction

Let us assume that the hypothesis is true for inputs of length m , Let xa be a string of length $m+1$ with a in Σ , then

$$\delta'(q_0', xa) = \delta'(\delta'(q_0', x), a)$$

By the inductive hypothesis ,

$$\delta'(q_0', x) = [P_1, P_2, P_3, \dots, P_j]$$

If and only if ,

$$\delta(q_0, x) = [P_1, P_2, P_3, \dots, P_j]$$

But by the definition of δ' ,

$$\delta'([P_1, P_2, P_3, \dots, P_i], a) = [r_1, r_2, r_3, \dots, r_k]$$

If and only if,

$$\delta([P_1, P_2, P_3, \dots, P_i], a) = [r_1, r_2, r_3, \dots, r_k]$$

Thus,

$$\delta'(q_0', xa) = [r_1, r_2, r_3, \dots, r_k]$$

If and only if,

$$\delta(q_0, xa) = [r_1, r_2, r_3, \dots, r_k]$$

Which establishes the inductive hypothesis,

To complete the proof, we have only to add that $\delta'(q_0, x)$ is in F' exactly when $\delta(q_0, x)$ contains a state of Q that is F .

Thus,

$$L(M) = L(M')$$

6. REGULAR EXPRESSION

The languages accepted by the finite automata are easily described as simple expression or regular expression.

Let ' Σ ' be the alphabet. The regular expression over Σ and the set they denote are defined recursively as follows,

- a) ϕ is the regular expression and denotes the empty set.
- b) ϵ is the regular expression and denotes the set $\{\epsilon\}$.
- c) For each 'a' in Σ , a is a regular expression and denotes the set $\{a\}$.
- d) If r and s are regular expression denoting languages R and S respectively then $(r+s)$, (rs) and (r^*) are regular expression that denote the set $R \cup S, RS, R^*$ respectively.

7. REGULAR EXPRESSION THEOREM

Statement

Let 'r' be the regular expression then there exists an NFA with ϵ -transition that accepts $L(r)$.

To Prove

$$L(M) = L(r).$$

That is the regular expression to a language 'L' can be accepted by automata with ϵ -moves.

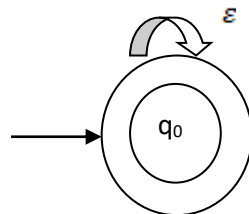
Proof

We can show by induction on the number of operators in the regular expression 'r' that is a NFA 'M' with ϵ -transitions, having one final state and no transition out of this final state.

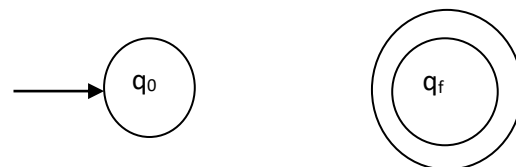
Basis: (Zero Operators)

The expression 'r' must be ϵ , ϕ or a for some a in Σ . The NFA for each of the expressions are,

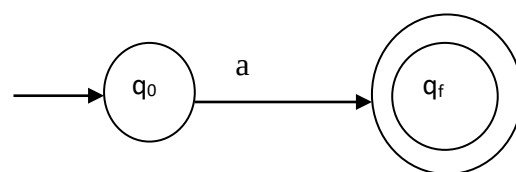
a) $r = \epsilon$



b) $r = \phi$



c) $r = a$



Hence if 'r' is the regular expression then there exists NFA accepting 'r'.

Induction

Assume that the theorem is true for regular expression with fewer than 'i' operators, that is $i \geq 1$. Let r have 'i' operators. There are three cases depending on the form of 'r'.

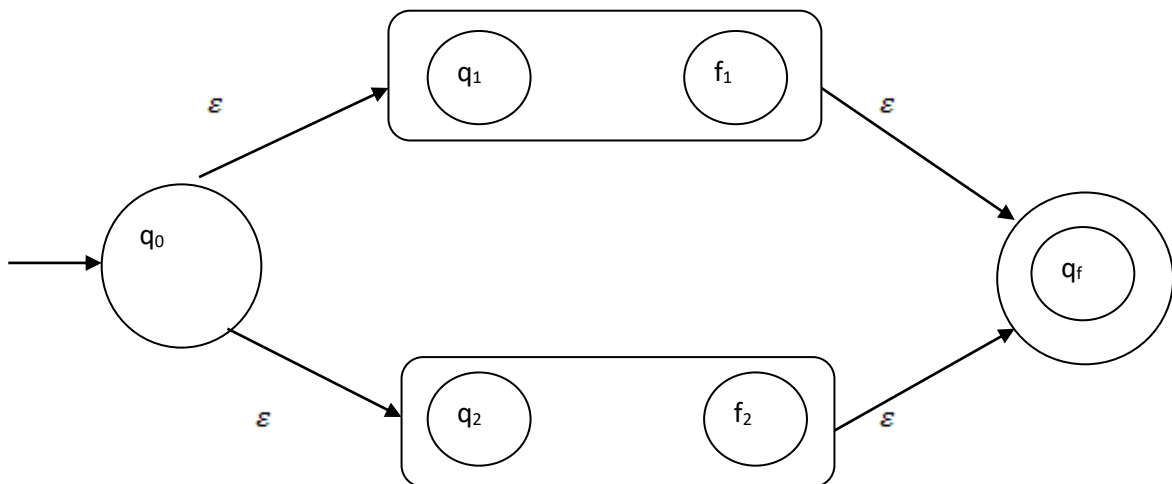
Case 1:

Union($r=r_1+r_2$)

Both r_1 and r_2 must have fewer than i operators. Thus there are NFA's $M_1=(Q_1,\Sigma_1,\delta_1,q_1,\{f_1\})$ and $M_2=(Q_2,\Sigma_2,\delta_2,q_2,\{f_2\})$ with $L(M_1)=L(r_1)$ and $L(M_2)=L(r_2)$. Since we may rename states of a NFA, we may assume Q_1 and Q_2 are disjoint. Let ' q_0 ' be a new initial state and f_0 a new final state.

Construct,

$$M=(Q_1 \cup Q_2 \cup \{q_0, f_0\}, \Sigma_1 \cup \Sigma_2, \delta, q_0, \{f_0\})$$



1. $\delta(q, \epsilon) = \{q_1, q_2\}$
2. $\delta(q, a) = \delta_1(q, a)$ For q in $Q_1 - \{f_1\}$ and a in $\Sigma_1 \cup \{\epsilon\}$
3. $\delta(q, a) = \delta_2(q, a)$ For q in $Q_2 - \{f_2\}$ and a in $\Sigma_2 \cup \{\epsilon\}$
4. $\delta(f_1, \epsilon) = \delta(f_2, \epsilon) = \{f_0\}$

We can recall the inductive hypothesis that there are no transitions out of f_1 or f_2 in M_1 or M_2 , thus the moves of M_1 and M_2 are present in M . The construction of M is depicted in the figure. Any path in the transition diagram of M from q_0 to f_0 must begin by going to either q_1 or q_2 on reading ϵ . If the path goes to q_1 , it may follow any path M_1 to f_1 and then goes to f_0 on

ϵ . Similarly, the path that begin by going to q_2 may follow any path in M_2 to f_2 and then go to f_0 on reading ϵ . It follows immediately that there is a path labelled x in M from q_0 to f_0 , if and only if there is a path labelled x in M_1 from q_1 to f_1 or path in M_2 from q_2 to f_2 . Hence $L(M) = L(M_1) \cup L(M_2)$ is derived.

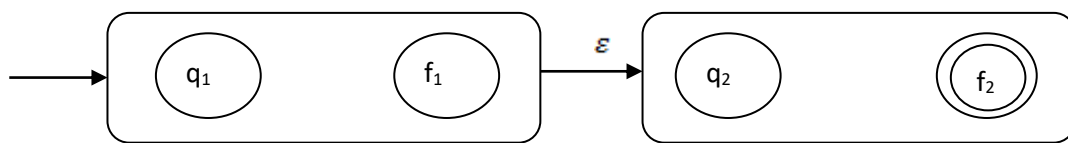
Case2:

Concatenation($r=r_1.r_2$)

Consider two NFA's $M_1=(Q_1, \Sigma_1, \delta_1, q_1, \{f_1\})$ and $M_2=(Q_2, \Sigma_2, \delta_2, q_2, \{f_2\})$ with $L(M_1)=L(r_1)$ and $L(M_2)=L(r_2)$.

Then,

$M=(Q_1 \cup Q_2, \Sigma_1 \cup \Sigma_2, \delta, q_1, \{f_2\})$



1. $\delta(q, a) = \delta_1(q, a)$ For q in $Q_1 - \{f_1\}$ and a in $\Sigma_1 \cup \{ \epsilon \}$
2. $\delta(q, a) = \delta_2(q, a)$ For q in $Q_2 - \{f_2\}$ and a in $\Sigma_2 \cup \{ \epsilon \}$
3. $\delta(f_1, \epsilon) = \{q_2\}$

Every path in M from q_1 to f_2 is a path labelled by string x from q_1 to f_1 followed by the edge from f_1 to q_2 labelled ϵ , followed by a path labelled by some string y from q_2 to f_2 .

Thus ,

$L(M) = \{xy/x \text{ in } L(M_1) \text{ and } y \text{ in } L(M_2) \text{ and } L(M) = L(M_1).L(M_2)\}$ is derived.

Case3:

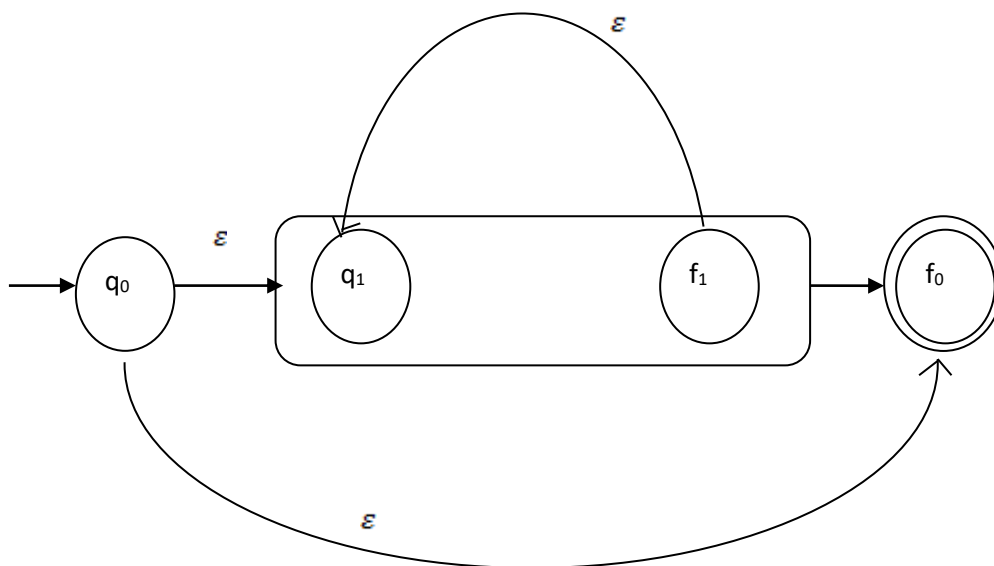
Kleene closure ($r=r_1^*$)

Let $M_1=(Q_1,\Sigma_1,\delta_1,q_1,\{f_1\})$ and $L(M_1)=L(r_1)$,Construct

$M=(Q_1 \cup \{q_0, f_0\}, \Sigma_1, \delta, q_0, \{f_0\})$

Where δ is given by,

$\delta(q_0, \epsilon) = \delta(f_1, \epsilon) = \{q, f_0\}$



$$\delta(q,a) = \delta_1(q,a) \text{ For } q \text{ in } Q_1 - \{f_1\} \text{ and } a \text{ in } \Sigma_1 \cup \{ \epsilon \}$$

Any path from q_0 to f_0 consists of either of a path from q_0 to f_0 on ϵ , followed by some number of paths from q_1 to f_1 then back to q_1 on ϵ each labelled by a string in $L(M_1)$ followed by a string in $L(M_1)$ followed by a path from q_1 to f_1 on the string in $L(M_1)$ from f_0 on ϵ . Thus there is a path in M from q_0 to f_0 on ϵ . Thus there is a path in M from q_0 to f_0 labelled x if and only if we can write $x = x_1 x_2 x_3 \dots x_j$, Such that x_j is in $L(M_1)$ then $L(M) = L(M^*)$.

5. TWO-WAY FINITE AUTOMATA

We have viewed the deterministic finite automata as a control unit that reads a tape moving one square right at each move. We needed non-determinism to the model which allowed many copies of control unit to exit and scan the tape simultaneously. Next we added ϵ -transitions which allowed change of state without reading the input symbol or moving the tape head. Next interesting extension is to allow the tape head with the ability to move left as well as right, such a finite automaton is called as two-way finite automaton.

The two-way finite automata accepts, the input symbol where tape head must move left or right at each move. A two finite automaton (2DFA) is a quintuple.

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where,

M is the NFA with ϵ moves.

Q is the finite set of non-empty states.

Σ is the finite set of non-empty symbols.

δ is the finite set of transitions (represented by transition table).

q_0 is the initial state.

F is the final state.

The transition mapping is given by

$$Q \times \Sigma \rightarrow Q \times (L, R).$$

- a) If $\delta(q,a)=(P,L)$ then in state q , scanning input symbol a , the 2DFA enters state 'p' and moves the head left one square.
- b) If $\delta(q,a)=(P,R)$,the 2DFA enters state 'p' and moves its head right one square .We introduce notation called “Instantaneous Description (ID)” of a 2DFA which the

input string, current state and position of the input head. Then we introduce the relation Γ_n on ID's such that $I_1 \Gamma_n I_2$ if and only if M can go from the instantaneous description I_1 to I_2 in one move.

An ID of M is a string in $\Sigma^* Q \Sigma^*$.The ID “ wqx ” where w and x are in Σ^* and q is in Q where ,

- 1) Wx is the input string.
- 2) Q is the current state,
- 3) The input head is scanning the first symbol of x

we define,

$$L(n)=\{w/q_0w \Gamma^* wp \text{ for some path } p \text{ in } f\}$$

That is, w is accepted by M , if starting in state q_0 with w on the input tape and the head at the left end of w . M eventually enters a final state at the same time it falls of the right end of the input tape.

For example

Consider a 2DFA that has the transition table of

Transition Table

δ	a	b
q_0	(q_0, R)	(q_1, R)
q_1	(q_1, R)	(q_2, L)
q_2	(q_0, R)	(q_2, L)

Consider the input 101001 .Since q_0 is the initial state the first ID is q_0 101001. So,

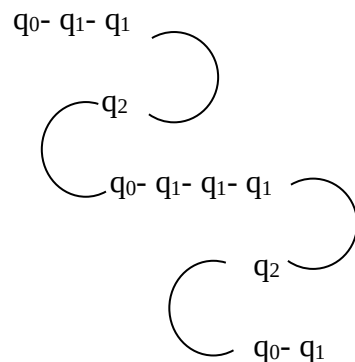
q_0 101001	Υ 1 q_1 01001	(Since $\delta(q_0, 1) = (q_1, R)$)
	Υ 10 q_1 1001	(Since $\delta(q_1, 0) = (q_1, R)$)
	Υ 1 q_2 01001	(Since $\delta(q_1, 1) = (q_2, L)$)
	Υ 10 q_0 1001	(Since $\delta(q_2, 0) = (q_0, R)$)
	Υ 101 q_1 001	(Since $\delta(q_2, 0) = (q_0, R)$)
	Υ 1010 q_1 01	(Since $\delta(q_1, 0) = (q_1, R)$)
	Υ 10100 q_1 1	(Since $\delta(q_1, 0) = (q_1, R)$)
	Υ 1010 q_2 01	(Since $\delta(q_1, 1) = (q_2, L)$)
	Υ 10100 q_0 1	(Since $\delta(q_2, 0) = (q_0, R)$)
	Υ 101001 q_1	(Since $\delta(q_0, 1) = (q_1, R)$)

Thus the string is accepted by two-way finite automata.

CROSSING OVER SEQUENCE

A useful picture of the behaviour of 2DFA consists of the input, the path followed by the head, and the state each time the boundary between two tape squares is crossed , with the assumption that the control enters its new state prior to moving head .For example ,the behaviour of the 2DFA M of 101001 is shown as

1	0	1	0	0	1
---	---	---	---	---	---



The list of states below each boundary between squares is termed as crossing sequence. Note that if 2DFA accepts its input, no crossing sequence may have represented state with the head moving in the same direction, otherwise the 2DFA being deterministic would be in loop thus never fall off the right end.

Another important observation about crossing sequences is that the first time boundary is crossed, the head must be moving right. Subsequent crossings must be in alternate directions. Thus odd-numbered elements of a crossing sequence represent right moves and even –

numbered elements represent left moves. If the input is accepted, it follows that crossing sequence are of odd length.

A crossing sequence $q_1, q_2, \dots, q_k$ is said to be valid if it is of odd length, and no two odd – and no two even numbered elements are identical. A 2DFA with 's' states can have valid crossing sequences of length at most $2s$, so the number of valid crossing sequence is finite. There are two types of crossing sequence

- a) Right –matching sequence.
- b) Left –matching sequence.

6. FINITE AUTOMATA WITH OUTPUT

The finite automata can have binary output. There are two distinct approaches,

- a) Moore machine
- b) Mealy machine

MOORE MACHINE

If the output is associated with each and every state, then it is called Moore machine

MEALY MACHINE

If the output is associated to each and every transition then it is called Mealy machine.

Both the machine takes the same input and returns the similar output

MOORE MACHINE

The Moore machine consists of six tuples,

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$$

Q is the finite set of non-empty states.

Σ is the finite set of non-empty input symbols.

Δ is the finite set of non-empty output symbols.

δ is the finite set of input transitions.

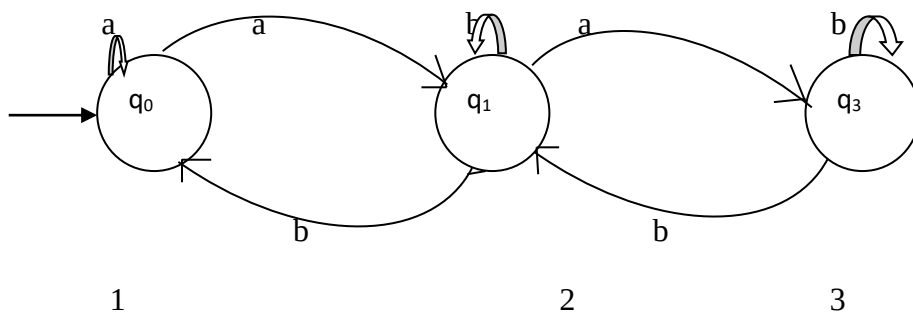
λ is the output function mapping.

q_0 is the initial state.

δ is the transition function from $Q \times \Sigma$ into Q is the function mapping Q into Δ and ' q_0 ' is the initial state.

e.g.

Consider a Moore machine,



Here the output function $\lambda(t)$ depends on ,

$$\lambda(t) = \lambda(q(t))$$

This is Moore machine because the output function depends on the present state and is independent of current output of the automaton.

$$\lambda: Q \rightarrow \Delta$$

MEALY MACHINE

The value of the output function $\lambda(t)$ is the more general case is a function of the present state $q(t)$ and present input $x(t)$.

That is,

$$\lambda(t) = \lambda(q(t), x(t)).$$

$$M = (Q, \Sigma, \Delta, \delta, \lambda, q_0,)$$

Q is the finite set of non-empty states.

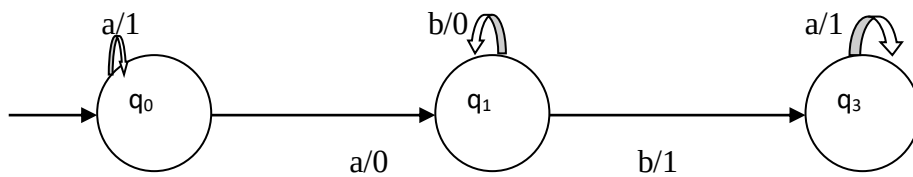
Σ is the finite set of non-empty input symbols.

Δ is the finite set of non-empty output symbols.

δ is the finite set of input transitions.

λ is the output function mapping.

q_0 is the initial state.



Here,

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

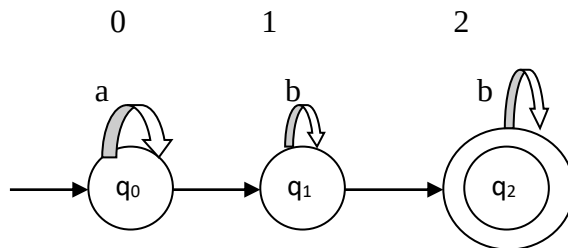
$$q_0 = \{q_0\}$$

$$\Delta = \{0, 1\}$$

Since each and every transition contains an output alphabet. This automata with output is called Mealy machine.

7. CONVERSION OF MOORE TO MEALY MACHINE

Consider a Moore machine



here,

$$M = (Q, \Sigma, \delta, \Delta, q_0, \lambda)$$

Then

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$q_0 = \{q_0\}$$

$$\Delta = \{0, 1, 2\}$$

$$\lambda(q_0) = 0$$

$$\lambda(q_1) = 1$$

$$\lambda(q_2) = 2$$

Transition Table

Δ	a	b
q_0	$\{q_0\}$	$\{q_1\}$
q_1	$\{q_2\}$	$\{q_1\}$

q_2	ϕ	$\{q_2\}$
-------	--------	-----------

Converting to Mealy:

$$\lambda(\delta(q_0, a)) = \lambda(q_0) = 0$$

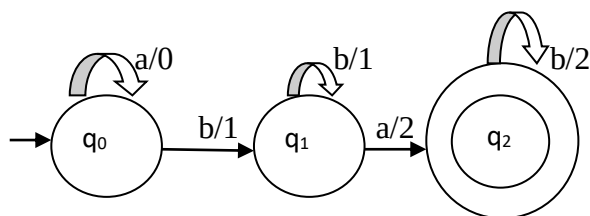
$$\lambda(\delta(q_0, b)) = \lambda(q_1) = 1$$

$$\lambda(\delta(q_1, a)) = \lambda(q_2) = 2$$

$$\lambda(\delta(q_1, b)) = \lambda(q_1) = 1$$

$$\lambda(\delta(q_2, a)) = \lambda(q_2) = 2$$

The final Mealy Transition diagram:



8.APPLICATIONS OF FINITE AUTOMATA

There are varieties of software design related problems that are simplified by automatic type conversion of regular expression notation to an efficient computer implementation to the corresponding finite automaton.

Two such applications are,

- Lexical Analyser.
- Text Editor.

LEXICAL ANALYSER

The tokens of the programming languages are almost with expression expressible as regular sets.

E.g. ALGOL identifiers, which are upper or lower case followed by any sequence of letter and digits with no limit on length, may be expressed as

$$(\text{Letter}) (\text{letter} + \text{digit})^*$$

Where letter stands for A+B+.....+Z+ a + b+.....+z and digits stand for 0+1+....+9.

FORTRAN identifiers length limit six and letters restricted to uppercase and symbol \$ may be expressed as

$$(\text{letter}) (\epsilon + \text{letter} + \text{digit})^5$$

Where “letter” now stands for (\$+A+B+.....Z)

A number of lexical –analyzer generators take as input a sequence of regular expression describing the tokens and produce a finite automaton recognizing any token. Usually they convert the regular expression to an NFA with ϵ –transitions. Each final state indicates the particular token is found, so the automaton is really a Moore machine. The lexical analyser produced by the generator is a fixed program that interprets coded tables together with the particular table that represents the finite automata recognizing the token. The Lexical Analysis is the first phase of a compiler.

TEXT EDITOR

Certain text editors and similar programs permit the substitution of a string of any string matching a given regular expression.

E.g. The UNIX text editor allows a command such as

S/bbb*/b/

This substitute a single blank for the first string of two or more blanks found in a given line. Let “any” denote the expression $a_0 + a_1 + \dots + a_m$, where a_i ‘s are all of a computer’s character except the “newline” character. We could convert a regular expression r to a DFA that accepts any $*r$. Note the presence of any $*$ allows us to recognise a member of $L(r)$ beginning anywhere in the line.

Actually what happens in the UNIX text editor is that the regular expression $*r$ is converted to an NFA with ϵ –transition and NFA is stimulated directly.